

LABORATORY

LAB 12

Information Display on TFT

Displaying Text and Basic Graphics Using the TFT Device API with
EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Coordinate System and Pixels	2
2.2	RGB565 Color	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Default TFT Device Configuration	4
3.3	TFT Connection	4
4	EduFramework API	5
4.1	Initializing the TFT with <code>TFT_Begin()</code>	5
4.2	Background Color and Graphics Primitives	6
4.3	Displaying Text	6
5	Lab Exercise	7
5.1	Requirements	7
5.2	Program	7
5.3	Program Explanation	8
5.4	Verification	9
6	Extension	9
6.1	Backlight Control	9
6.2	Displaying Numeric Values	10
6.3	Creating Custom Colors with <code>TFT_Color565()</code>	10
6.4	Querying Display Dimensions	10
6.5	Address Window in the TFT Controller	11
6.6	Context-Based Advanced API	11
6.7	Extension Exercise	12
7	References	12

1 Introduction

1.1 Lab Overview

A TFT display allows an embedded system to present information using graphical elements such as pixels, text, lines, and basic geometric shapes. Each pixel is defined by a position within the display area and a color value; from these basic elements, software can build visual interfaces for system status, sensor data, or control information [1].

This lab focuses on using a TFT display through the EduFramework Device API. Instead of working directly with the command set of the display controller, the application uses high-level APIs to initialize the display, configure text color and position, and draw graphics primitives.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the pixel coordinate system and how graphical elements are positioned on a TFT display.
2. Explain 16-bit color representation using the RGB565 format.
3. Describe the default TFT Device configuration in EduFramework, including the display area dimensions and control Logical Pins.
4. Use `TFT_Begin()`, text APIs, and graphics APIs to display information on a TFT screen.
5. Recognize the extended capabilities of the TFT Device API, including backlight control, numeric output, custom colors, and the context-based Advanced API.

2 Background

2.1 Coordinate System and Pixels

A graphical display is organized as a grid of pixels. In the coordinate system used in this lab, the pixel at the upper-left corner has coordinate $(0, 0)$, the x coordinate increases from left to right, and the y coordinate increases from top to bottom. Each graphical element is defined using the corresponding pixel coordinates [2]. The coordinate system can be visualized as follows:

```
(0,0) -----> x
|
|
|
|
v
y
```

An individual pixel is identified by a coordinate pair (x, y) . A line uses two endpoints, while a rectangle uses the coordinates of its starting corner together with its width and height. Graphics primitives such as pixels, lines, rectangles, and circles provide the basis for building more complex graphical elements [1].

Note

Valid coordinates depend on the current display dimensions. When drawing graphics, the application should choose coordinates and dimensions that keep each element within the intended display area.

2.2 RGB565 Color

A common way to represent color on a TFT display is to use 16 bits for each pixel. In the RGB565 format, 5 bits are allocated to the red component, 6 bits to green, and 5 bits to blue [1].

The bit layout can be represented as follows:

```

15          11 10          5 4          0
+-----+-----+-----+
| Red - 5 bit | Green - 6 bit | Blue - 5 bit |
+-----+-----+-----+
```

EduFramework provides commonly used predefined color constants:

```

TFT_BLACK
TFT_WHITE
TFT_RED
TFT_GREEN
TFT_BLUE
TFT_YELLOW
TFT_CYAN
TFT_MAGENTA
```

The graphics APIs receive these 16-bit color values to determine the color of the pixel or graphical element being drawn [6].

The ST7789 supports the 16-bit RGB565 pixel-data format in addition to other color modes. EduFramework uses RGB565 for the TFT Device API color interface [3] [6].

3 Hardware Setup**3.1 Required Hardware**

The lab exercise uses the MaaZEDU Development Board and an LCD TFT module supported by the current EduFramework configuration. MaaZEDU is a development board based on the S32K144 microcontroller [5].

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
LCD TFT module	TFT display using a controller compatible with the current EduFramework implementation.
Jumper wires	Connect power, SPI, and control signals between the TFT and MaaZEDU.
USB Cable	Connect the board to the computer for power and program upload.

Table 12.3.1. Hardware used in the lab exercise

3.2 Default TFT Device Configuration

The EduFramework Beginner API uses a default hardware configuration so that the application can initialize the display using only `TFT_Begin()`. The current configuration uses a `240 x 280` pixel display area, `xOffset = 0`, `yOffset = 20`, and the control Logical Pins shown in the table below [6].

Parameter	Default	Role
Width	<code>240</code> pixels	Display-area width.
Height	<code>280</code> pixels	Display-area height.
X Offset	<code>0</code>	Starting column offset in controller memory.
Y Offset	<code>20</code>	Starting row offset in controller memory.
CS	<code>GPI00</code>	TFT Chip Select.
DC	<code>GPI01</code>	Selects command or display data.
RST	<code>GPI02</code>	TFT hardware reset.
BLK	<code>GPI03</code>	Backlight control.

Note

These values are the default Beginner API configuration in the current EduFramework version. When the hardware uses different pins or dimensions, the Advanced API allows the application to provide a custom configuration.

3.3 TFT Connection

The display uses `SPI_SCK` as the clock signal and `SPI_SOUT` as the data line from the MCU to the TFT. In the default configuration, the `CS`, `DC`, `RST`, and `BLK` signals use `GPI00` through `GPI03`, respectively [6].

TFT	MaaZEDU	Function
VCC	<code>3.3V</code>	Power supply for the TFT module.
GND	<code>GND</code>	Common ground between the module and the board.
SCL / SCK	<code>SPI_SCK</code>	SPI clock.
SDA / MOSI	<code>SPI_SOUT</code>	SPI data from the MCU to the TFT.
CS	<code>GPI00</code>	Chip Select.
DC	<code>GPI01</code>	Data / Command select.
RES / RST	<code>GPI02</code>	Hardware Reset.
BLK / BL	<code>GPI03</code>	Backlight control.

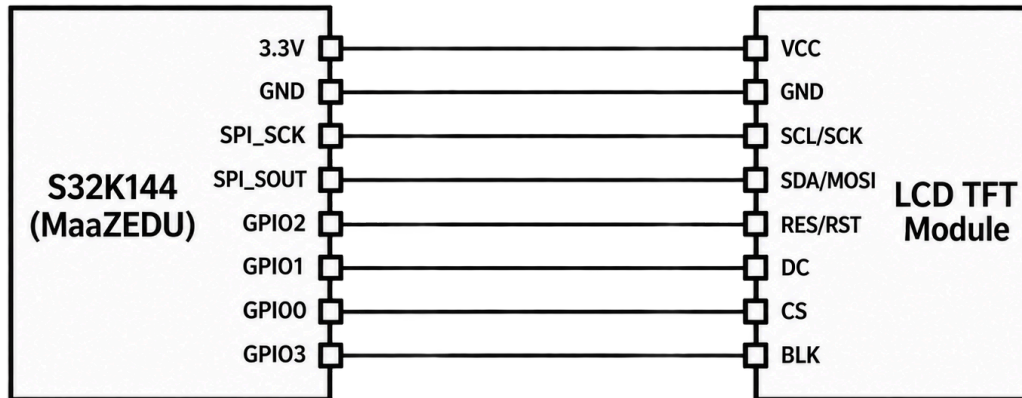


Figure 12.3.1. LCD TFT connection diagram with MaaZEDU Development Board

Note

The `SDA` label on the TFT module in this lab refers to the `SPI MOSI` data line, not the `SDA` signal of I2C communication. The TFT does not use the `SPI_SIN / MISO` line in this lab exercise.

4 EduFramework API

The EduFramework TFT Device API organizes display operations into initialization, text rendering, and graphics-primitives groups. The Beginner API uses an internal context and the default hardware configuration, so the application does not need to manage the ST7789 controller command sequence directly [6].

4.1 Initializing the TFT with `TFT_Begin()`

`TFT_Begin()` initializes the TFT Device using the default EduFramework configuration. The API prepares SPI communication, the control Logical Pins, and the display context before the application performs graphical operations [6].

`TFT_Begin`

Initialize the TFT Device using the default EduFramework configuration.

Syntax

```
TFT_Begin();
```

Return value

`true` when the TFT is initialized successfully; `false` when initialization does not complete successfully.

Example:

```
if (true == TFT_Begin())
{
    TFT_FillScreen(TFT_BLACK);
}
```

4.2 Background Color and Graphics Primitives

`TFT_FillScreen()` fills the entire display area with one color. The graphics-primitives APIs allow the application to work with pixels, lines, rectangles, and circles. Similar primitives are commonly used in TFT graphics libraries to construct higher-level display elements [1] [6] .

API	Function
<code>TFT_FillScreen(color)</code>	Fill the entire display with one color.
<code>TFT_DrawPixel(x, y, color)</code>	Draw one pixel at the specified coordinate.
<code>TFT_DrawLine(x0, y0, x1, y1, color)</code>	Draw a line between two points.
<code>TFT_DrawRect(x, y, width, height, color)</code>	Draw a rectangle outline.
<code>TFT_FillRect(x, y, width, height, color)</code>	Draw a filled rectangle.
<code>TFT_DrawCircle(x, y, radius, color)</code>	Draw a circle using its center and radius.
<code>TFT_FillCircle(x, y, radius, color)</code>	Draw a filled circle.

4.3 Displaying Text

The TFT Device maintains the cursor position, text color, background color, and text size. After these properties are configured, `TFT_Print()` or `TFT_Printfn()` uses the current state to render a string on the display [6] .

API	Function
<code>TFT_SetCursor(x, y)</code>	Set the starting position for the next text output.
<code>TFT_SetTextColor(color)</code>	Set the character color.
<code>TFT_SetTextBackground(color)</code>	Set the character background color.
<code>TFT_SetTextSize(size)</code>	Set the font scale factor.
<code>TFT_Print(text)</code>	Display a string at the current cursor position.
<code>TFT_Printfn(text)</code>	Display a string and move the cursor to the next line.

Example:

```
TFT_SetCursor(20U, 30U);
TFT_SetTextColor(TFT_YELLOW);
TFT_SetTextBackground(TFT_BLACK);
```

```
TFT_SetTextSize(3U);  
TFT_Println("EduFramework");
```

Note

The font is managed internally by the TFT Device implementation. The application does not need to access the font bitmap directly to perform basic text operations.

5 Lab Exercise

5.1 Requirements

Build a program that displays a static interface on the TFT using the EduFramework Beginner API. The interface consists of two lines of text, one separator line, one outlined rectangle, one filled rectangle, one outlined circle, and one filled circle.

The program must use the default EduFramework TFT configuration and must not call the `ST7789_*` functions directly in the main lab exercise.

5.2 Program

In `src/main.c`, implement the hardware-verified program as follows:

```
#include "Arduino.h"
#include "lcd_tft.h"

int main(void)
{
    setup();

    if (false == TFT_Begin())
    {
        while (1)
        {
            /* TFT initialization failed. */
        }
    }

    TFT_FillScreen(TFT_BLACK);

    TFT_SetCursor(20U, 30U);
    TFT_SetTextColor(TFT_YELLOW);
    TFT_SetTextBackground(TFT_BLACK);
    TFT_SetTextSize(3U);
    TFT_Println("EduFramework");

    TFT_SetCursor(20U, 75U);
    TFT_SetTextColor(TFT_CYAN);
    TFT_SetTextSize(2U);
    TFT_Println("S32K144 TFT");

    TFT_DrawLine(20U, 105U, 220U, 105U, TFT_WHITE);

    TFT_DrawRect(20U, 125U, 80U, 50U, TFT_GREEN);
    TFT_FillRect(140U, 125U, 80U, 50U, TFT_BLUE);

    TFT_DrawCircle(60U, 220U, 25U, TFT_RED);
    TFT_FillCircle(180U, 220U, 25U, TFT_MAGENTA);

    while (1)
    {
        /* Display remains unchanged. */
    }
}
```

Listing 12.5.1. Program for displaying text and basic graphics on the TFT

5.3 Program Explanation

`setup()` initializes the underlying EduFramework components. `TFT_Begin()` then initializes the TFT Device using the default configuration; if initialization fails, the program remains in the error loop and does not perform any further display operations [6].

After the Device is ready, `TFT_FillScreen(TFT_BLACK)` creates a black background across the entire display area. The text APIs configure the cursor position, text color, background color, and text size before the two strings "EduFramework" and "S32K144 TFT" are rendered.

The following instructions use graphics primitives to create a separator line, two rectangles, and two circles. `TFT_DrawRect()` and `TFT_DrawCircle()` draw only the outlines, while `TFT_FillRect()` and `TFT_FillCircle()` fill the complete internal area with the specified color [6].

After all content has been drawn, the program does not need to update the display further. The final loop keeps the application running while the content already written to the TFT remains visible.

5.4 Verification

Build and upload the program to the MaaZEDU Development Board. Observe the display after the board starts. The display should show a black background with two lines of text at the top. Below the text is a white horizontal line, followed by a green outlined rectangle and a blue filled rectangle. At the bottom, a red outlined circle and a magenta filled circle are displayed.

The expected TFT output is illustrated below:

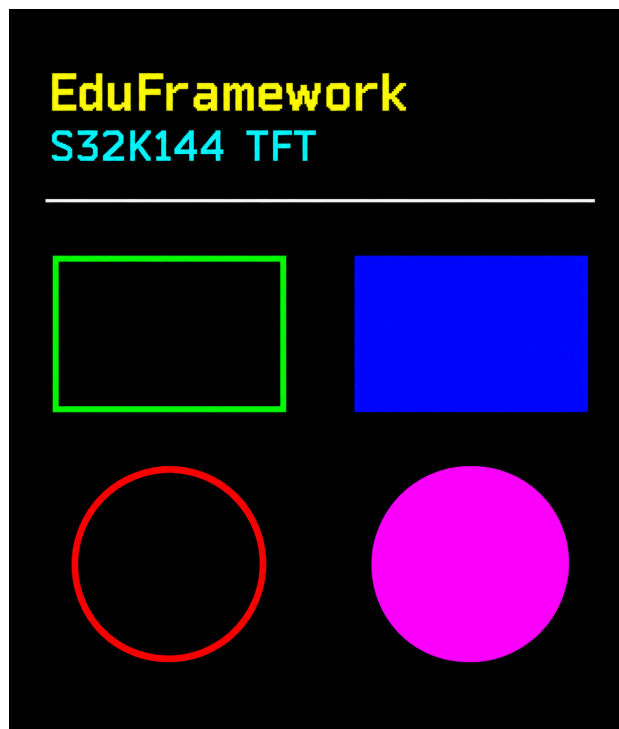


Figure 12.5.1. Expected output on the TFT display.

EXPECTED RESULT

The TFT is initialized successfully and displays both lines of text together with all elements at the intended positions. The colors, positions, and fill styles of the elements correspond to the parameters passed to the TFT Device API.

6 Extension

The Beginner API provides common operations using the default TFT configuration. In addition to the APIs used in the main exercise, the TFT Device also supports backlight control, numeric output, custom RGB565 colors, display-dimension queries, and a context-based Advanced API for other hardware configurations [6].

6.1 Backlight Control

The backlight can be controlled directly using:

```
TFT_BacklightOn();  
TFT_BacklightOff();
```

In the default configuration, the backlight signal uses GPIO3 [6].

These APIs allow the application to turn off the backlight when the display is not needed and turn it on again when interaction with the display is required.

6.2 Displaying Numeric Values

In addition to character strings, the Beginner API provides functions for displaying integer and floating-point values:

API	Function
<code>TFT_PrintInt(value)</code>	Display an integer value.
<code>TFT_PrintlnInt(value)</code>	Display an integer value and move to the next line.
<code>TFT_PrintFloat(value, decimals)</code>	Display a floating-point value with the selected number of decimal places.
<code>TFT_PrintlnFloat(value, decimals)</code>	Display a floating-point value and move to the next line.

Example:

```
TFT_SetCursor(20U, 30U);
TFT_SetTextColor(TFT_WHITE);
TFT_SetTextBackground(TFT_BLACK);
TFT_SetTextSize(2U);

TFT_Print("Temperature: ");
TFT_PrintFloat(23.56F, 2U);
```

These APIs are suitable for applications that display sensor data because the application does not need to convert numeric values to strings before rendering them [6].

6.3 Creating Custom Colors with `TFT_Color565()`

`TFT_Color565()` converts red, green, and blue color components into a 16-bit RGB565 value used by the graphics APIs. Each input component is represented in the range 0 to 255, after which the framework packs the values into RGB565 format [6].

Example:

```
uint16_t orange;

orange = TFT_Color565(255U, 128U, 0U);

TFT_FillRect(20U, 20U, 80U, 40U, orange);
```

This API allows colors to be created beyond the set of predefined color constants.

6.4 Querying Display Dimensions

The application can read the current TFT Device dimensions using:

```
uint16_t width;
uint16_t height;

width = TFT_Width();
height = TFT_Height();
```

`TFT_Width()` and `TFT_Height()` help the code avoid direct dependence on the values 240 and 280, especially when the application is used with another display configuration [6].

6.5 Address Window in the TFT Controller

A TFT controller does not necessarily need to update the entire display for every operation. A rectangular region can be defined by column and row limits, and subsequent pixel data are written into the selected region. This arrangement allows only the area that needs to change to be updated instead of rewriting the complete frame [2].

The ST7789 provides the `CASET` command to set the column range, `RASET` to set the row range, and `RAMWR` to begin writing pixel data into frame memory [3].

The EduFramework Advanced API exposes this mechanism through:

```
ST7789_SetAddressWindow(&tft, x0, y0, x1, y1);
```

In typical applications, the framework graphics APIs handle this operation internally; the application only needs to call `ST7789_SetAddressWindow()` directly when developing low-level graphics functionality or implementing a custom optimization.

6.6 Context-Based Advanced API

The Beginner API manages a default TFT context and uses fixed pin, dimension, and offset configuration. When the application requires a different configuration, the Advanced API allows a separate `ST7789_t` context to be created [6].

Example initialization:

```
ST7789_t tft;  
  
ST7789_Init(  
    &tft,  
    GPIO0,  
    GPIO1,  
    GPIO2,  
    240U,  
    280U,  
    0U,  
    20U  
);
```

Listing 12.6.1. Initializing the TFT with the Advanced API

The parameters correspond to the context, `CS`, `DC`, `RST`, `width`, `height`, `xOffset`, and `yOffset`. The application can change these values when the TFT hardware uses a different connection or display area [6].

The main graphics APIs at the Advanced level include:

Advanced API	Role
ST7789_Init()	Initialize a TFT context using the selected configuration.
ST7789_SetAddressWindow()	Set the frame-memory region for pixel-write operations.
ST7789_DrawPixel()	Draw one pixel.
ST7789_DrawLine()	Draw a line.
ST7789_DrawRect()	Draw a rectangle outline.
ST7789_FillRect()	Draw a filled rectangle.
ST7789_FillScreen()	Fill the entire display area.
ST7789_DrawCircle()	Draw a circle.
ST7789_FillCircle()	Draw a filled circle.
ST7789_DrawChar()	Draw one character using the specified position and properties.
ST7789_DrawString()	Draw a string using the specified position and properties.

Note

The Beginner API uses the generic `TFT_*` naming scheme, while the current Advanced API directly exposes the `ST7789_*` implementation. Therefore, another display can use this Advanced API only when its controller and control method are compatible with the current ST7789 implementation.

6.7 Extension Exercise

Build a simple information screen using the Beginner API to:

- create a custom color using `TFT_Color565()` ;
- display an integer using `TFT_PrintInt()` ;
- display a floating-point value using `TFT_PrintFloat()` ;
- use `TFT_Width()` and `TFT_Height()` instead of hard-coding the display dimensions;
- turn the backlight off for a period of time and turn it on again using the backlight APIs.

Then select one graphical element and try to implement it again using the Advanced API with a separate `ST7789_t` context.

EXPECTED RESULT

The display correctly presents the numeric data and custom color, the display dimensions are obtained from the TFT Device instead of being hard-coded, the backlight can be turned on or off through the API, and one graphics primitive is implemented successfully using an `ST7789_t` context.

7 References

- [1] Cornell University, *TFT LCD Display* , ECE 4760 - Designing with Microcontrollers.
- [2] University of Wisconsin-Madison, *ILI9341 LCD Controller* , ECE353 - Introduction to Microprocessor Systems.
- [3] Sitronix Technology Corporation, *ST7789V - 240RGB x 320 dot 262K Color with Frame Memory Single-Chip TFT Controller/Driver* , Rev. 1.3, 2014.

- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [5] MaaZEDU, *MaaZEDU Development Board Guide*, Rev. 1.0, 2025.
- [6] EduFramework, *LCD TFT Device API*, EduFramework Source Code.